Investigating the Performance of Policy-Gradient Reinforcement Learning Algorithms for Quadrotor Control Task in MuJoCo Simulation

Candidate Number: 282862

May 2026

1 Introduction

Autonomous quadrotor control presents a challenging task as it requires continuous control decisions to maintain stable flight behaviour. Deep Reinforcement Learning (DRL) enables agents to learn through interactions with an environment and reward feedback. By updating neural network parameters through backpropagation, agents can learn complex behaviours that maximise cumulative rewards. This learning capability makes DRL suitable for complex control tasks such as autonomous quadrotor navigation and control (Azar et al., 2021) .

Training in simulation plays a crucial role in the development of autonomous systems, as agents typically require extensive exploration during training. Such exploration can result in unstable or unsafe behaviour, creating a risk of damage to physical hardware. Therefore, training in simulation before transferring acquired knowledge to real-world systems is a common and practical approach in robotics (Chan et al., 2024).

This project investigates and compares two Reinforcement Learning algorithms for drone control in a simulated environment: REINFORCE, an early policy-gradient method, and Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017).

The study aims to investigate the limitations of REINFORCE in achieving stable and effective control behaviour for the drone agent. Building on these observations, the study further explores whether modifications to REINFORCE can improve training stability and performance.

2 Background work

Both algorithms investigated in this project belong to the class of policy-gradient methods, which optimise policies directly through interactions with an environment.

REINFORCE is an early foundational policy-gradient algorithm that suffers from high variance during training, motivating the development of many subsequent reinforcement learning approaches and optimisation techniques. Despite its limitations, REINFORCE established principles that continue to influence modern policy-gradient methods.

Recent work outside autonomous control has explored extensions to traditional REINFORCE methods. Hu et al. (Hu et al., 2025) introduced REINFORCE++, an extension of the traditional REINFORCE algorithm developed for Reinforcement Learning from Human Feedback (RLHF) in Large Language Models. The approach demonstrated that incorporating additional optimisation techniques can improve learning performance. This suggests REINFORCE can set a strong foundation if further improved through modifications.

Furthermore, Philip Becker-Ehmck et al. (Becker-Ehmck et al., 2020) explored reinforcement learning for autonomous drone flight using a model-based framework. They used REINFORCE with a reparameterization trick for training a drone flight controller. However, the approach also incorporated a model-based component, where the flight controller acted as a critic for trajectory evaluation. This indicates that REINFORCE can be applied in autonomous system development, although it is often combined with additional techniques or integrated into architectures tailored to specific system constraints and requirements.

PPO is an Actor–Critic algorithm proposed to improve the stability of policy optimisation by limiting large policy updates (Schulman et al., 2017). PPO has demonstrated strong training stability and has been successfully applied to complex tasks such as autonomous drone racing (Song et al., 2021). However, the algorithm still has limitations, including the introduction of bias through its clipping objective and high computational requirements.

3 Research question

Although previous studies on training drone controllers have clearly highlighted the limitations of REINFORCE, particularly its high variance, this study aims to investigate these limitations in practice. The aim of this report is to identify the limitations of REINFORCE and investigate why, although it can perform effectively in certain applications, it may demonstrate reduced performance for continuous control tasks when compared to the more stable PPO approach.

Additionally, the research questions of this report include:

1. How can simple modifications to REINFORCE reduce training instability in continuous-control tasks?
2. To what extent can simple improvements to REINFORCE reduce the performance gap between REINFORCE and PPO?
3. What are the key differences in learning stability, convergence, and overall performance between REINFORCE, improved REINFORCE, and PPO?

4 Methods

4.1 Simulation

The simulation is implemented in MuJoCo using an XML file. The agent is represented as a simplified drone model consisting of a central body, four propeller arms, and four motor actuators.

The agent is placed inside a top-open box with dimensions of approximately 20 × 20 MuJoCo units to assist with reward shaping, e.g. collisions with the walls immediately terminate the episode. The agent's initial position is set at the centre of the box, corresponding to the starting location (0,0,0.3)

The model uses gravity of -9.81 in the z-axis and a timestep of 0.005 seconds. The agent has four motor actuators, which apply upward thrust forces to the drone body through propeller thrust sites. The actuator control values are constrained within a range of 0 to 100, limiting the amount of thrust that can be generated by each motor (1, lines 69–75).

4.2 Policy Network

This report section will explain the policy network architecture. It is only applied in the implementation of the REINFORCE-based approach, as PPO uses its own policy network architecture provided by Stable-Baselines3.

The 13-dimensional state representation obtained from the environment is used as the input to this neural network. The architecture consists of an 13-value input layer, three hidden layers with ReLU activation with 64 neurons each, and an output layer with a sigmoid activation function. The hidden layers allow the network to learn non-linear relationships between the state variables and motor actions.

The sigmoid activation constrains the output values to the range [0,1]. They are scaled by a factor of 175, which was selected as a tuning hyperparameter to produce suitable thrust values for the drone simulation (3, lines 24-32).

4.3 Environment

The environment is designed to be compatible with Gymnasium-based algorithms by following the required environment structure. This allows the implementation of PPO from the Stable-Baselines3 library.

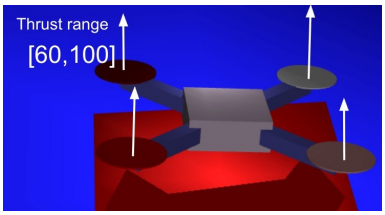
4.3.1 Action Representation

To produce enough lift, approximately 80 units of force should be applied. To make the environment suitable for PPO, the action space is defined with a lower bound of 0 and an upper bound of 100.

Although a custom neural network can produce values greater than 100, the actions within the step

function are clamped between 60 and 100. This is done to reduce sudden changes from very high thrust values to very low thrust values, which can lead to aggressive and unstable behaviour. Such large variations initially caused the drone to behave unpredictably and increased the difficulty of training, with the lower limit of 60 also acting as a safety threshold.

These actions are applied to the MuJoCo model through the motor actuators, after which the simulation advances for a predefined number of simulation steps. The environment then updates and returns the new observation state, calculates the reward based on the actions taken at the each step of the episode.



$$P = [x, y, z]$$
$$q = [q_w, q_x, q_y, q_z]$$
$$v = [v_x, v_y, v_z]$$
$$\omega = [\omega_x, \omega_y, \omega_z]$$

Figure 1: State and action representation of the drone agent.

4.3.2 State Representation

For each environment step, the agent receives the current state observation and outputs four values that can then be thrust processed by simulation and interpreted as thrust forces for the propellers. As presented in *Figure 1*, the observation consists of 13 values such as position, angular orientations, linear and angular velocities.

4.3.3 Reward Function

The reward function was designed to encourage the drone to learn stable hovering behaviour and maintaining a target height of 1.0 in a z-axis. Instead of rewarding only the final objective, multiple reward components are used to gradually guide the learning process, and it can be considered that most of these rewards are quite immediate. The main part of the reward function is based on the drone's vertical position: the agent receives a positive reward when it moves closer to the target height and a penalty when it moves further away. Extra rewards are given when the drone remains close to the desired height, particularly within a small distance threshold, to encourage stable hovering rather than simply reaching the target. (4, lines 100-121)

Several penalties are included to prevent unstable or unrealistic behaviour. For example, if the drone flies too high, remains too low for an extended period, flips over, spins excessively, or collides with the walls, the episode is terminated and a negative reward is applied. These conditions help prevent the agent from exploiting the reward function and learning a bad policy, but also intends to make learning quicker, by terminating episodes that can be considered unsuccessful early.

The reward function also considers the behaviour of the motors. Sudden changes in motor values are penalised and a small reward is also applied when the motor outputs remain relatively balanced.(4, lines 135-151)

Horizontal movement away from the centre of the environment, large angular velocities, and unstable vertical movement receive penalties. The orientation of the drone is also considered, with rewards for maintaining an upright position and penalties when the drone begins to tilt excessively. The drone is also encouraged to stay close to the centre location(4, lines 154-222)

Finally, the reward value is clipped between -10 and 10 before being returned. This prevents very large reward values from dominating training and helps improve learning stability. Overall, the reward function combines several smaller objectives to gradually guide the agent towards learning controlled and stable hovering behaviour. It also gives the agent a small additional reward for simply staying alive, which may be considered the only long-term-oriented component of the reward function.

4.4 PPO

For this project, the PPO implementation provided by the Stable-Baselines3 library was used.

PPO is commonly implemented using an Actor–Critic architecture. In this framework, two neural networks are used during training. The Actor network is responsible for selecting actions based on the current state of the environment, while the Critic network estimates the value of the current state. The Critic provides feedback to the Actor through advantage estimates, allowing the policy to improve over time.

The clipping mechanism constrains the ratio between the probability of taking an action under the new policy and the probability under the old policy. If this ratio changes beyond a predefined threshold, represented by the parameter ϵ, the objective function clips the ratio and limits further updates.

For autonomous quadrotor control, small changes in motor outputs can significantly affect the behaviour of the drone. Therefore, PPO is well suited for this task because its constrained updates allow the agent to learn effective control strategies while reducing instability during training.

4.5 REINFORCE

REINFORCE is a Monte Carlo policy-gradient algorithm that learns by updating a policy directly based on complete episode returns. The main idea behind REINFORCE is to update the policy by increasing the probability of actions associated with higher rewards, while decreasing the probability of actions resulted in lower rewards.(1)

$$\nabla_\theta E_{q_\theta(z)}[f(z)] = E_{q_\theta(z)}\left[f(z)\nabla_\theta \log q_\theta(z)\right] \qquad (1)$$

In this project, the policy network outputs the mean values of a Normal distribution for four continuous actions corresponding to the drone motors. Actions are then sampled from this distribution to introduce stochastic behaviour and encourage exploration of the environment.

The standard deviation of the Normal distribution controls the degree of exploration. Larger standard deviation values increase randomness in the selected actions, which may help the agent discover potentially better strategies. However, excessive exploration can also introduce instability into training, causing inconsistent behaviour and reducing convergence speed. Therefore, a balance between exploration and stability is required.

The probability of selected actions is represented using the log probability of the sampled actions under the current policy distribution. These log probabilities are later used in the policy-gradient update to reinforce actions that result in higher returns.

The discounted return is calculated at the end of each episode using a discount factor of 0.99, and the returns are additionally normalised. After the policy loss has been calculated, backpropagation is performed to update the network parameters to improve future action selection.

5 Experiments

The experimentation section was designed to implement a number of optimisations to the basic formula of REINFORCE.

The improvements for this REINFORCE were partly inspired by REINFORCE++ . Although the algorithms were used in the context of Large Language Models, the authors suggest that it can also be applied to complex multi-step reasoning tasks. The Clipping objective was intended to be inherited from the PPO.

The reparameterisation trick was used to separate the stochastic noise component from the

network output during action sampling. This does not directly improve learning stability, but instead makes the sampling process differentiable, allowing gradients to flow through the sampling operation during backpropagation. This can reduce gradient variance.

Instead of using a separate critical network to calculate advantages, normalized returns are used as an approximation. The discounted returns are centered by subtracting the mean and scaled by dividing by the standard deviation. With the aim to reduce variance during training, it provides a relative measure of whether an action performed better or worse than the average behaviour within the episode. This effectively acts as a baseline and produces normalized advantage values. If an advantage value is positive, it indicates that the action performed well, and its probability should be increased. Conversely, a negative advantage indicates that the action performed worse than expected and its probability should be reduced.

The policy loss incorporates a clipping mechanism inspired by PPO. Rather than allowing unrestricted policy updates, the probability ratio between the current and previous policy is constrained within a specified range. This helps prevent large updates that may destabilize training while still allowing the policy to improve more gradually. (6, lines 92 - 110)

6 Results

6.1 Learning PPO

The learning process under PPO successfully achieved its primary objective. A demonstration of the trained agent can be viewed here: https://youtube.com/shorts/3ev-ldHn4Ds

The trained agent was able to generate sufficient lift, maintain stable hovering behaviour around the target height, and partially recover from external disturbances. .

In the example shown in the video, the drone was subjected to external forces to evaluate its ability to recover and maintain stability. While the agent can recover to some extent, it can still easily crash under stronger disturbances, indicating an area for further improvement.

The learning process under PPO can be divided into three stages:

1. Discovering lift, staying still for many timesteps
2. Making many mistakes during exploration
3. Eventually learning to stabilise flight

Although the first stage could potentially be improved through additional constraints, such as restricting the action space to values that generate sufficient lift, the results suggest that the agent gradually learns that maintaining stability is one of the most highly rewarded behaviours when standing still.

Alternatively, the agent was also observed exploiting weaknesses within the reward function. For example, it learned behaviours such as flying near the edges of the environment while avoiding collisions and still receiving relatively high rewards throughout the episode. Additional reward shaping was introduced to reduce this behaviour.

Over many millions of timesteps, the learning performance improved progressively. This was partly due to modifications to the reward function, as the initial reward design was overly aggressive while the control task itself remained highly complex. This lad to the reward graph not being clearly defined.

6.2 REINFORCE vs REINFORCE+

Due to its high variability, continuous drone control under REINFORCE appears to be unsuccessful. After implementation, the graphs show high variance because the gradients are too noisy, as shown in Figure 3. This means that rewards frequently point in different directions, and very small changes can significantly affect the learning trajectory.

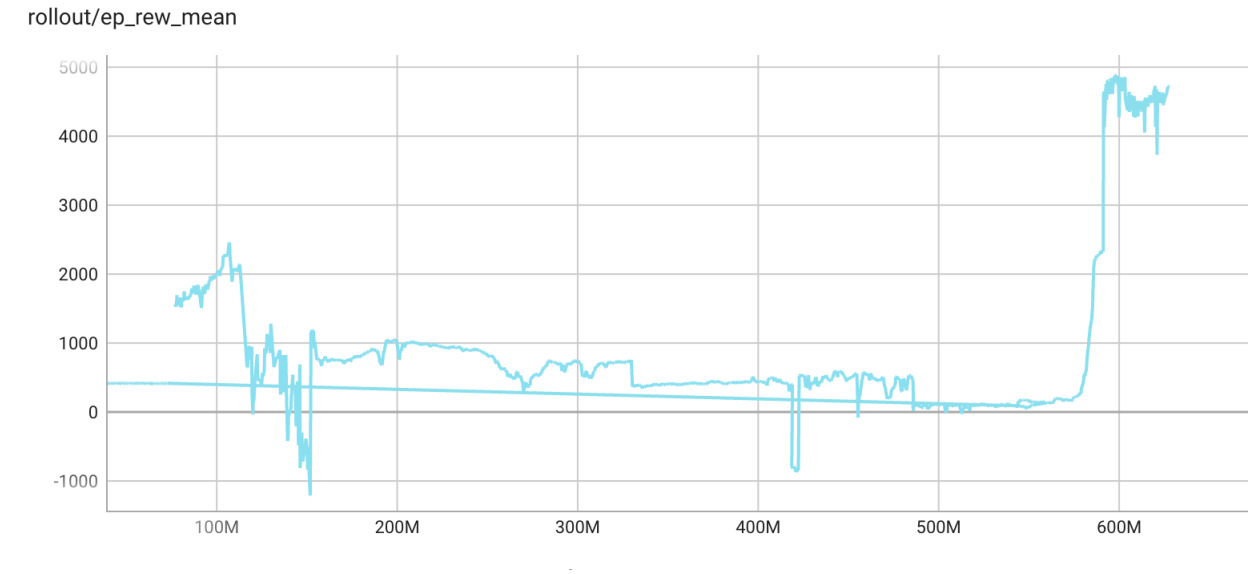


Figure 2: PPO training (reward/timesteps)

In principle, an episode consists of multiple steps, where each step can be either good or bad. Accordingly, after each episode, when REINFORCE is applied and backpropagation is performed, the weights are updated to increase the probability of good actions and decrease the probability of poor actions. Since these updates happen simultaneously, improving the average reward per episode becomes difficult and unstable. Simply speaking, the agent can learn a good policy, but it can also forget it almost immediately. The learning behaviour is highly unstable (Figure 3 and 4).

Furthermore, because REINFORCE relies on complete episode returns rather than estimating future rewards using a value function, the updates are strongly affected by short-term fluctuations and random exploration. This makes the learning process highly sensitive to individual trajectories and can slow convergence in continuous-control tasks such as drone stabilisation.

REINFORCE+ demonstrated comparatively more stable learning behaviour within the experimental setup. As presented in Figure 3, the reward trend stays positive fpr longer. To conclude, the introduction of advantage normalisation and PPO-inspired clipping mechanisms helped reduce the magnitude of unstable updates.

7 Evaluation

The results demonstrate that PPO significantly outperformed basic REINFORCE in learning stability and convergence for continuous drone control. While basic REINFORCE exhibited unstable learning due to high-variance policy updates, the modified REINFORCE implementation showed improved stability through advantage normalisation and PPO-inspired clipping mechanisms. However, a considerable performance gap remained between REINFORCE-based approaches and PPO. These findings suggest that although REINFORCE can be improved through relatively simple modifications, algorithms incorporating more stable policy update

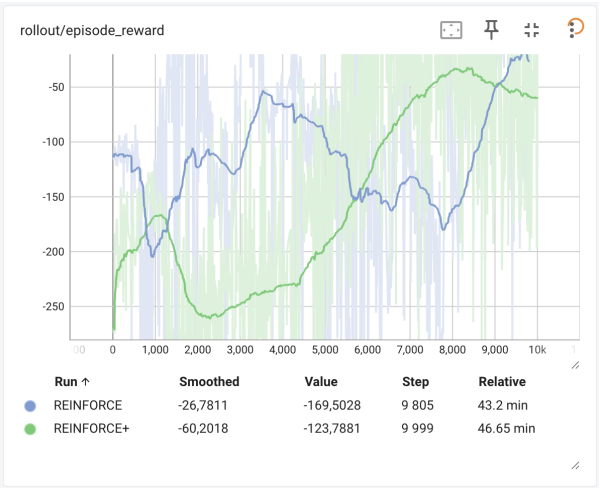


Figure 3: Smoothed episode reward comparison between REINFORCE and improved REINFORCE

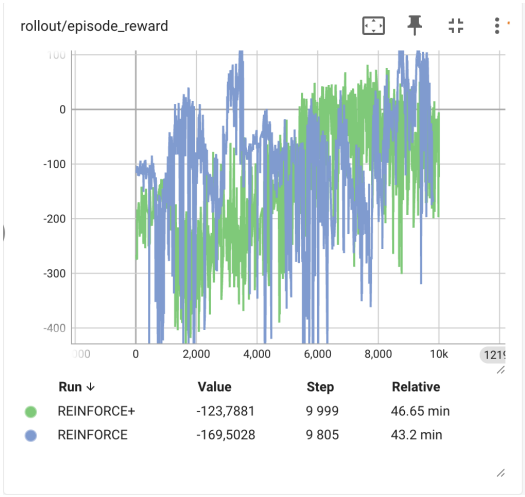


Figure 4: Raw episode reward progression during training

mechanisms remain better suited for continuous-control tasks.

The main focus of the evaluation is learning stability and how quickly the policy network is able to improve and achieve the desired behaviour. Figure 3 and Figure 4 demonstrate the poor and moderate convergence of basic REINFORCE and improved REINFORCE respectively, while PPO achieved a considerably better level of convergence.

To evaluate the results, an important difference between PPO and REINFORCE must first be addressed: when and how the policy is updated. For REINFORCE, the policy is updated only at the end of an episode after the full trajectory has been collected. Since complete episode returns are used, policy updates can be strongly affected by noisy rewards and unstable trajectories. As a result, the policy may briefly learn beneficial behaviours but can also quickly forget them due to unstable updates.

In contrast, PPO updates the policy using batches of collected experiences rather than relying solely on complete episodes. Although performance can still be evaluated using episodes, PPO primarily uses timesteps as the measure of training progress. PPO also incorporates additional mechanisms that reduce update variance, contributing to more stable learning behaviour.

This suggests that improving how policy updates are performed may provide a possible direction for optimisation of REINFORCE-based methods. Rather than relying entirely on complete episode returns, distributing updates in a more stable manner could reduce training instability.

A further limitation of this work was insufficient computational power. Training PPO required approximately 50–70 hours. During development, limitations in the reward function design also required repeated reward shaping and parameter tuning to guide the agent towards desirable behaviours. While reward shaping is relatively common in reinforcement learning, the learned policy proved highly sensitive to reward function design changes, making training more difficult at times.

There are several factors that may have simpli-

fied the learning process, including reducing the size of the environment, experimenting with episode lengths, and introducing more sophisticated reward function designs. Furthermore, the physical environment itself appears somewhat simplified and unstable, while the drone representation could also be improved.

8 Future work

In a real drone controller, control is not represented as four independent upward forces. Instead, navigation is typically controlled through pitch, roll, yaw, and thrust commands. A more realistic representation of these control inputs could improve realism and make the simulation more representative of real-world drone systems.

Additionally, quadrotor dynamics involve strong interactions between motors, where coordinated thrust adjustments across propellers influence the overall motion and stability of the drone. This may motivate alternative network designs and control architectures.

As with any simulation-based approach, it would be highly valuable to further investigate the design of the agent and identify potential weaknesses that could affect the transfer of learned behaviour to a real-world drone system. Addressing the reality gap remains a fundamental challenge in reinforcement learning and autonomous robotic systems.

Therefore, a significant future improvement would involve incorporating sensors and a richer world representation, while also introducing partially observable environments. If the drone receives incomplete information from the environment, it may still learn to infer missing information from previous observations and interactions.

A partially observable framework would improve realism. For example, wind information could remain hidden from the agent. This would better reflect real-world robotic systems, where environmental disturbances are not directly available as explicit variables but instead influence behaviour indirectly through changes in system dynamics and movement (R).

Importantly, it would also be valuable to investigate modular approaches involving multiple specialised networks for tasks such as stabilisation, target navigation, and altitude control. Rather than learning all behaviours simultaneously within a single policy, separate components could be trained for individual subtasks and later integrated into a larger control system.

References

Azar, A., Koubaa, A., Ahmed, N., Ibrahim, H., Fathy, Z., Kazim, M., Ammar, A., Benjdira, B., Khamis, A., Hameed, I., and Casalino, G. (2021). Drone deep reinforcement learning: A review. *Electronics*, 10.

Becker-Ehmck, P., Karl, M., Peters, J., and van der Smagt, P. (2020). Learning to fly via deep model-based reinforcement learning.

Chan, J. H., Liu, K., Chen, Y., Sagar, A. S. M. S., and Kim, Y.-G. (2024). Reinforcement learning-based drone simulators: survey, practice, and challenge. *Artificial Intelligence Review*.

Hu, J., Liu, J. K., Xu, H., and Shen, W. (2025). Reinforce++: Stabilizing critic-free policy optimization with global advantage normalization.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Song, Y., Steinweg, M., Kaufmann, E., and Scaramuzza, D. (2021). Autonomous drone racing with deep reinforcement learning.